

COMPILER DESIGN - ASSESSMENT

ANSWERS

Name: S. Jaxon Rizal

Reg. No: 192511421

Q1. Intermediate Code Generation & Control Flow

Code Fragment:

```
int sum=0;
for (i = 0; i<=10; i++)
    sum=sum+a[i];
```

(a) Generation of Three Address Code (TAC)

Assuming 4-byte integers for array indexing, the generated TAC is as follows:

- (1) `sum = 0` // (i) Initialization of variables
- (2) `i = 0`
- (3) `if i <= 10 goto (5)` // (ii) Evaluation of loop condition
- (4) `goto (10)`
- (5) `t1 = i * 4` // (iii) Array access operations
- (6) `t2 = a[t1]`
- (7) `sum = sum + t2` //(iii) Assignment operation
- (8) `i = i + 1` // (iv) Increment operation
- (9) `goto (3)` // End of loop
- (10) `...`

(b) Basic Blocks and Control Flow Graph (CFG)

(i) Identification of leader statements:

- Statement (1): First statement is always a leader.
- Statement (5): Target of the conditional jump at (3).
- Statement (10): Target of the unconditional jump at (4).

- Statement (3): Target of the unconditional jump at (9).

(ii) Formation of basic blocks:

- **B1:** Statements (1) to (2)
- **B2:** Statements (3) to (4)
- **B3:** Statements (5) to (9)
- **B4:** Statement (10)

(iii) Construction of the Control Flow Graph (CFG):

- **B1 → B2:** Execution naturally flows from initialization to the loop condition.
- **B2 → B3:** Condition is true ($i \leq 10$).
- **B2 → B4:** Condition is false, loop terminates.
- **B3 → B2:** End of loop body, jump back to evaluate the condition.

Q2. TAC and Target Code Generation

Assignment:

```
x := (a+b)*(c-d) + ((e/f)*(a+b))
```

(a) Generation of Three Address Code (TAC)

- **(i) Identification of sub-expressions:** `(a+b)` is evaluated twice. We can reuse it.
- **(ii & iii) Use of temporary variables and order of evaluation:**

```
t1 = a + b    // Common sub-expression
t2 = c - d
t3 = t1 * t2  // Left part of the addition
t4 = e / f
t5 = t4 * t1  // Right part of the addition using reused t1
t6 = t3 + t5
x = t6
```

(b) Target Code Generation

Applying a simple code generation algorithm using registers R0, R1, and R2:

```

LD R0, a
ADD R0, b      // R0 contains t1 (a+b)
LD R1, c
SUB R1, d      // R1 contains t2 (c-d)
MUL R1, R0     // R1 contains t3 (t1*t2). Left operand evaluated.
LD R2, e
DIV R2, f      // R2 contains t4 (e/f)
MUL R2, R0     // R2 contains t5 (t4*t1). Right operand evaluated.
ADD R1, R2     // R1 contains t6 (t3+t5)
ST x, R1      // Store final result in memory location x

```

Q3. DAG Construction and Optimized Code

Basic Block:

```

1. d := b * c
2. e := a + b
3. b := b * c
4. a := e - d

```

(a) Construction of the Directed Acyclic Graph (DAG)

(i) Nodes:

- Node 1: Leaf b_0
- Node 2: Leaf c_0
- Node 3: Leaf a_0
- Node 4: Operator $*$ with children Node 1 and Node 2. Labels: d, b
- Node 5: Operator $+$ with children Node 3 and Node 1. Label: e
- Node 6: Operator $-$ with children Node 5 and Node 4. Label: a

(ii & iii) Redundant Computation: The expression $b*c$ is computed once (Node 4) and used for both d and b, eliminating the redundant recalculation in statement 3.

(b) Optimized Code Generation using ONE Register

To avoid overwriting values needed later, we determine the sequence from the DAG. The final optimized 1-register code execution path is:

```

LD R0, a
ADD R0, b    // e = a+b
ST e, R0     // stored in memory
LD R0, b
MUL R0, c    // d = b*c
ST d, R0
ST b, R0     // b updated to d
LD R0, e
SUB R0, d    // a = e-d
ST a, R0

```

Q4. Optimal Target Code Generation

Arithmetic Expression:

$(a+b) - (c - (d+e))$

(a) Using TWO Registers

According to the Sethi-Ullman algorithm, we evaluate the tree based on register requirements. The right subtree $c - (d+e)$ requires more registers (2) than the left subtree $a+b$ (1).

```

// Evaluate right subtree: c - (d+e)
LD R0, d
ADD R0, e    // R0 = d+e
LD R1, c
SUB R1, R0   // R1 = c - (d+e)

// Evaluate left subtree: a+b
LD R0, a     // R0 can be safely reused
ADD R0, b    // R0 = a+b

// Final evaluation
SUB R0, R1   // R0 = (a+b) - (c - (d+e))

```

(b) Using ONE Register

With only one register, we must spill intermediate results to memory using a temporary variable (t1, t2).

```

// Evaluate inner sub-expression d+e
LD R0, d
ADD R0, e
ST t1, R0      // Store d+e to memory

// Evaluate c - (d+e)
LD R0, c
SUB R0, t1
ST t2, R0      // Store right subtree result to memory

// Evaluate a+b
LD R0, a
ADD R0, b      // R0 holds a+b

// Final subtraction
SUB R0, t2     // R0 = (a+b) - (c - (d+e))

```

Q5. Code Optimization Techniques

Original Code:

```

for (int i=0; i<n; i++) {
    a[i] = x * y + z;
    a[i+n] = x * y + w;
}

```

(a) Common Subexpression Elimination and Code Motion

- **(i) Identification:** `x * y` is calculated twice in each iteration.
- **(ii) Invariant Computation (Code Motion):** `x`, `y`, `z`, and `w` are loop invariants. Thus, `x * y + z` and `x * y + w` do not change across iterations. They can be hoisted outside the loop.
- **(iii) Optimized Code:**

```

int temp_xy = x * y;
int val1 = temp_xy + z;
int val2 = temp_xy + w;
for (int i = 0; i < n; i++) {
    a[i] = val1;
    a[i+n] = val2;
}

```

(b) Analysis of the Optimized Code

- **(i) Reduction in computations:** Originally, the loop performed $2n$ multiplications and $2n$ additions. The optimized code performs exactly 1 multiplication and 2 additions, entirely outside the loop.
- **(ii) Improvement in runtime efficiency:** The heavy arithmetic operations have been replaced by simple memory writes inside the loop body, reducing CPU cycles per iteration drastically.
- **(iii) Effect on loop performance:** This ensures optimal use of processor pipelines and cache performance. The time complexity factor for arithmetic drops from $O(n)$ to $O(1)$.